

CryptoQuant Terminal

Fase 1: Auditoria de Flujo Completo

Mapa completo del sistema desde la entrada de datos hasta la salida final

Fecha	2026-06-05
Version	1.0
Repo	github.com/coverdraft/cryptoquant-terminal
Ultimo commit	c9b027d
Metodologia	6 Fases (solo mapeo, sin modificaciones)
Estado	FASE 1 COMPLETADA

Indice

1. Resumen Ejecutivo
2. Arquitectura General del Sistema
3. Flujo de Datos Completo (6 Fases)
4. Catalogo de Modulos (89 archivos, ~40 singletons)
5. Mapa de Rutas API (112 endpoints)
6. Esquema de Base de Datos (38 modelos)
7. Integraciones Externas
8. Pipeline del Strategy Decision Engine (SDE)
9. Kill Switches y Gestion de Riesgo
10. Capital Allocation Pipeline
11. Estado de las Conexiones Criticas
12. Hallazgos y Observaciones

1. Resumen Ejecutivo

Este documento presenta el resultado de la **Fase 1: Auditoria de Flujo Completo** del sistema CryptoQuant Terminal. Se ha construido un mapa exhaustivo del sistema desde la entrada de datos hasta la salida final, identificando todos los modulos, dependencias, integraciones, flujos de datos, transformaciones y puntos de decision. **No se han realizado modificaciones al codigo** — esta fase es exclusivamente de mapeo y documentacion.

CryptoQuant Terminal es una plataforma de trading cripto semi-autonoma construida con Next.js 16.1.3 (Turbopack), Prisma 7, SQLite, Tailwind CSS y shadcn/ui. El sistema cuenta con 89 archivos de servicio organizados en 6 directorios, 112 endpoints API, 38 modelos de base de datos, y aproximadamente 40 singletons de servicio activos. La arquitectura sigue un pipeline de 6 fases: Ingesta de Datos, Analisis e Inteligencia, Estrategia y Validacion, Decision y Riesgo, Asignacion de Capital, y Ejecucion.

El componente central es el **Strategy Decision Engine (SDE)**, que actua como el cerebro del sistema. Implementa un pipeline de 6 pasos: vetos duros, scores compuestos, calidad de senal, estado + accion de capital, recomendacion de capital, y registro de auditoria. El SDE transforma la plataforma de una herramienta de analisis que produce informacion a un sistema de trading que produce decisiones accionables.

Metricas Clave del Sistema

Metrica	Valor
Archivos de servicio	89
Singletons activos	~40
Endpoints API	112
Modelos Prisma (DB)	38
Tablas con @@map	22
Indices compuestos	30+
Componentes UI (shadcn)	50
Pestanas activas en UI	19
Integraciones API externas	13
Metodos de capital allocation	16 (5 activos en v1)
Patrones de vela detectados	36 (5 timeframes)
Arquetipos de trader	8

2. Arquitectura General del Sistema

El sistema CryptoQuant Terminal sigue una arquitectura modular de 6 capas, donde cada capa depende de las anteriores pero no de las posteriores. Los datos fluyen de izquierda a derecha: desde las fuentes externas de datos, a traves de los motores de analisis e inteligencia, hacia los motores de estrategia y validacion, convergiendo en el SDE para la toma de decisiones, y finalmente hacia la asignacion de capital y la ejecucion (paper trading).

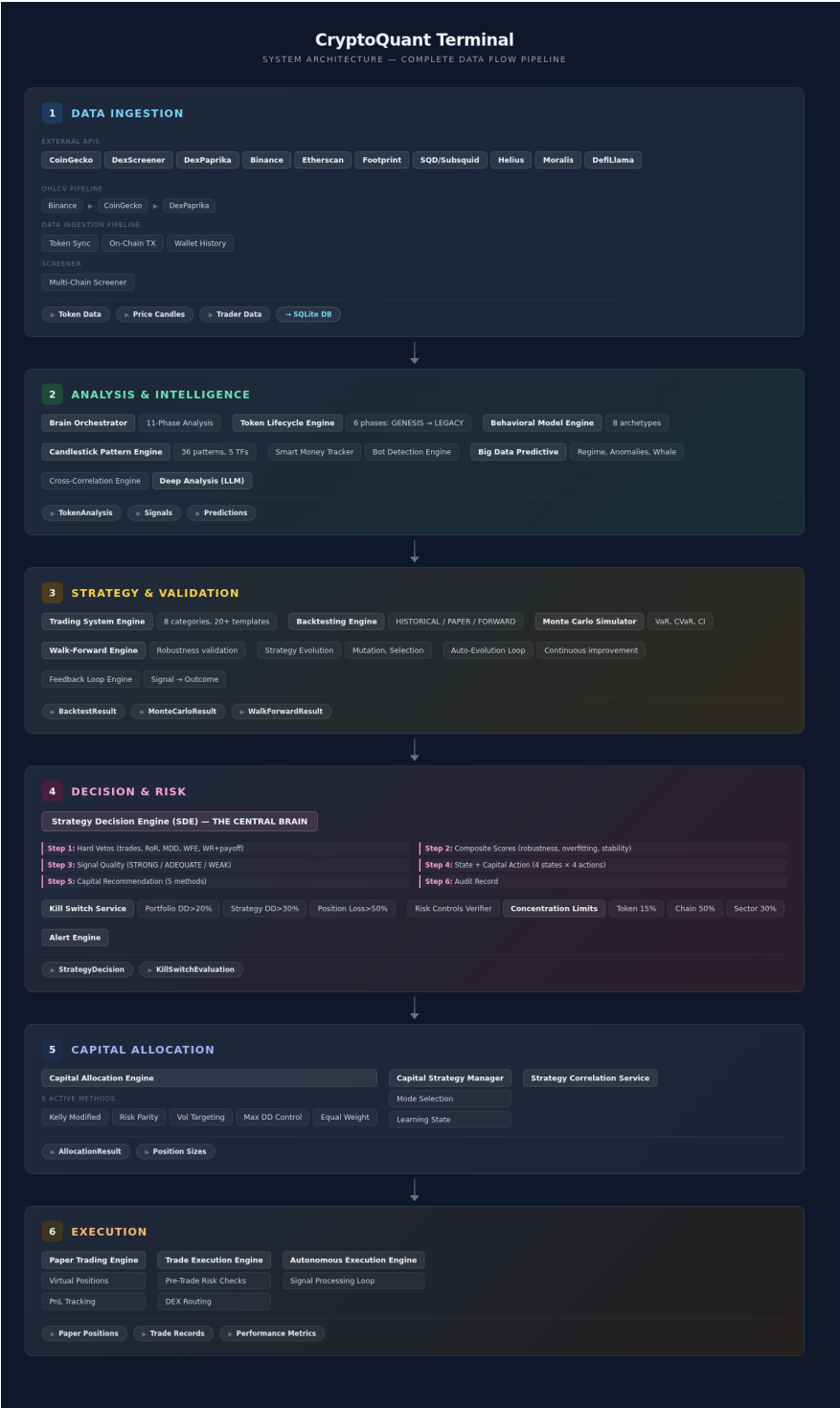


Figura 1: Flujo completo del sistema — Desde la ingesta de datos hasta la ejecucion

La arquitectura se organiza en 6 fases claramente diferenciadas. La Fase 1 (Ingesta) recopila datos de 13+ fuentes externas a través de clientes API especializados, con un sistema de cache unificado y cascada (Binance -> CoinGecko -> DexPaprika). La Fase 2 (Análisis) ejecuta un pipeline de 11 fases por token a través del Brain Orchestrator, produciendo análisis de ciclo de vida, modelos comportamentales, patrones de vela, y predicciones. La Fase 3 (Validación) ejecuta backtests, simulaciones Monte Carlo, análisis walk-forward, y evolución de estrategias. La Fase 4 (Decision) es el SDE — el cerebro central que sintetiza todos los datos en decisiones accionables. La Fase 5 (Asignación) calcula tamaños de posición usando 5 métodos activos. La Fase 6 (Ejecución) opera el paper trading con controles de riesgo integrados.

3. Flujo de Datos Completo

3.1 Fase 1: Ingesta de Datos

La ingesta de datos es la puerta de entrada del sistema. Se conecta con 13+ fuentes externas de datos a traves de clientes API especializados, cada uno con su propio rate limiting, cache, y manejo de errores. El componente central es el OHLCV Pipeline, que implementa una cascada de fuentes: primero intenta Binance (mayor cobertura y precision), luego CoinGecko (cobertura amplia), y finalmente DexPaprika (tokens DEX no listados en CEX).

- **OHLCV Pipeline:** Cascada Binance -> CoinGecko -> DexPaprika. Agrega timeframes superiores (1m->5m->15m->1h->4h->1d). Backfill historico con deteccion de gaps.
- **Data Ingestion Pipeline:** Orquesta sync de tokens (DexScreener), market data (CoinGecko), transacciones on-chain (Jupiter/Solana RPC/Ethereum RPC), wallet history, y smart money data.
- **Multi-Chain Screener:** Screening multi-cadena agregando datos de DexPaprika, CoinGecko y DexScreener.
- **Universal Data Extractor:** Framework de extraccion que envuelve Moralis, Helius, DefiLlama, EtherscanV2, y CryptoDataDownload.
- **Unified Cache:** Cache TTL en memoria con deduplicacion de requests, rate-limit awareness, y eviction policies por fuente.

Clientes API Especializados

Cliente	Funcionalidad	Cache
CoinGecko Client	Tokens paginados, OHLCV, market charts, global data, trending, search	TTL: 60s-5min
DexScreener Client	DEX pairs, token search, trending, liquidez	TTL: 30s-2min
DexPaprika Client	20+ chains, pools, swaps, OHLCV, buy/sell pressure, smart money swaps	TTL: 60s-5min
Binance Client	OHLCV candles, symbol info, tickers (intervalos 1m a 1M)	TTL: 15s-1min
Etherscan Client	Token transfers, transacciones, descubrimiento de traders	Rate-limited
Footprint Client	Token prices, OHLCV, protocol TVL, chain overviews	TTL: 5min
SQD/Subsquid Client	Indexer blockchain, eventos historicos, transfers	On-demand
Helius Client	Transacciones Solana, parsed transactions	TTL: 2min
Moralis Client	Multi-chain wallets, balances, NFTs	TTL: 5min
DefiLlama Client	Protocol TVL, yields, chain overview	TTL: 10min
EtherscanV2 Client	Multi-chain EVM data (via Etherscan V2)	Rate-limited
CryptoDataDownload	Historical CSV data	On-demand
Dune Client	SQL queries on blockchain data	On-demand

3.2 Fase 2: Analisis e Inteligencia

El Brain Orchestrator ejecuta un pipeline de 11 fases por token: data collection -> market context -> lifecycle -> behavior -> bot/whale detection -> operability -> predictive signals -> candlestick patterns -> deep analysis -> cross-correlation -> recommended action. El Brain Scheduler ejecuta 7 tareas periodicas: brain cycle (5min),

market sync (5min), OHLCV backfill (15min), signal validation (15min), evolution check (1hr), capital update (10min), y data cleanup (6hr). El scheduler persiste su estado en DB para sobrevivir reinicios.

Modulo	Funcion	Archivo
Brain Orchestrator	Pipeline de 11 fases por token, batch analysis, wallet profiling	brain-orchestrator.ts
Brain Scheduler	7 tareas periodicas, persistencia de estado, WS events	brain-scheduler.ts
Brain Cycle Engine	Loop SCAN->FILTER->MATCH->STORE->FEEDBACK->GROWTH 24/7	brain-cycle-engine.ts
Brain Analysis Pipeline	Orquesta todos los engines en secuencia, mantenimiento periodico	brain-analysis-pipeline.ts
Token Lifecycle Engine	6 fases: GENESIS->INCIPIENT->GROWTH->FOMO->DECLINE->LEGACY	token-lifecycle-engine.ts
Behavioral Model Engine	8 arquetipos, matrices 3D, actualizacion bayesiana	behavioral-model-engine.ts
Candlestick Pattern Engine	36 patrones, 5 timeframes, scoring por confluencia	candlestick-pattern-engine.ts
Deep Analysis Engine	Sintesis con LLM (z-ai-sdk) + fallback rule-based	deep-analysis-engine.ts
Cross-Correlation Engine	P(outcome conditions), tablas de probabilidad historica	cross-correlation-engine.ts
Smart Money Tracker	Deteccion de senales SM, perfiles de wallet, flujo SM	smart-money-tracker.ts
Bot Detection Engine	8 tipos de bots, 20+ senales, deteccion batch	bot-detection.ts
Big Data Predictive	Regimenes, anomalias, whale forecasting, mean reversion	big-data-engine.ts
Pattern Compression	Compresion de patrones con decaimiento temporal, limite 500/cat	pattern-compression-pipeline.ts
Token DNA Recalculator	Recalculo de scores DNA desde datos de trader intelligence	token-dna-recalculator.ts
Brain Capacity Engine	Metricas de capacidad en 12 categorias, niveles DORMANT->OPTIMAL	brain-capacity-engine.ts

3.3 Fase 3: Estrategia y Validacion

La capa de estrategia y validacion transforma los analisis en estrategias probadas. El Backtesting Engine simula estrategias contra datos historicos con 3 modos (HISTORICAL/PAPER/FORWARD), calculando metricas comprehensivas: Sharpe, Sortino, Calmar, max drawdown, win rate, profit factor, recovery factor, monthly returns, y phase breakdowns. El Monte Carlo Simulator genera miles de caminos aleatorios para calcular VaR, CVaR, intervalos de confianza, y distribuciones de max drawdown. El Walk-Forward Engine divide datos en ventanas de entrenamiento/validacion para probar robustez fuera de muestra. El Feedback Loop valida senales predictivas contra resultados reales y refina sistemas suboptimos. El Auto-Evolution Loop ejecuta ciclos continuos de evaluacion, mutacion, backtest y despliegue de estrategias mejoradas.

Modulo	Funcion	Archivo
Backtesting Engine	Simulacion historica, metricas completas, stop-loss/take-profit	backtesting-engine.ts
Monte Carlo Simulator	VaR, CVaR, intervalos de confianza, LCG PRNG reproducible	monte-carlo-simulator.ts
Walk-Forward Engine	Training/validation windows, robustez OOS, degradacion	walk-forward-engine.ts
Feedback Loop Engine	Validacion senal->resultado, refinacion, variantes sinteticas	feedback-loop-engine.ts
Strategy Evolution Engine	Mutacion, seleccion, prueba via backtest y WF	strategy-evolution-engine.ts
Auto-Evolution Loop	Ciclo continuo: underperformers -> mutacion -> backtest -> deploy	auto-evolution-loop.ts
Backtest Loop Engine	Loop automatizado por stages (EARLY/MID/STABLE)	backtest-loop-engine.ts

Modulo	Funcion	Archivo
Statistical Validation	t-tests, chi-square, correlacion, tamano muestral, decay temporal	statistical-validation.ts
Trading System Engine	8 categorias, 20+ templates, CRUD, phase configs	trading-system-engine.ts
Technical Indicators	SMA, EMA, RSI, MACD, Bollinger, ATR, Stochastic, ADX, VWAP, Vol Profile	technical-indicators.ts
Signal Generators	Smart money, rug pull, V-shape, liquidity trap, technical signals	signal-generators.ts
Parameter Drift Analyzer	Comparacion vs baseline, clasificacion STABLE->UNSTABLE	parameter-drift-analyzer.ts
Strategy State Manager	Maquina de estados: IDLE->SCANNING->ANALYZING->...->COOLING	strategy-state-manager.ts
Strategy Templates	8 categorias, seeding, templates built-in	strategy-templates.ts
Backtest Data Bridge	Carga datos reales al backtester, normalizacion, gaps	backtest-data-bridge.ts

3.4 Fase 4: Decision y Riesgo

El Strategy Decision Engine (SDE) es el cerebro central del sistema. Implementa un pipeline de 6 pasos que transforma datos de backtest, Monte Carlo, walk-forward, operability y paper trading en una decision accionable con 3 dimensiones: State (ACTIVE/CONDITIONAL/PAUSED/REJECTED), Capital Action (INCREASE/MAINTAIN/REDUCE/EXIT), y Signal Quality (STRONG/ADEQUATE/WEAK). Cada decision genera un registro de auditoria completo con snapshots de inputs, processing, decision y configuracion. El Kill Switch Service monitorea 7 niveles de riesgo: portfolio DD>20%, strategy DD>30%, position loss>50%, token concentration>15%, chain concentration>50%, sector concentration>30%, y emergency global pause.



Figura 2: Pipeline detallado del Strategy Decision Engine (SDE) — 6 pasos

Modulo	Funcion	Archivo
Strategy Decision Engine	Pipeline veto-first: vetos->scores->quality->state->capital->audit	strategy-decision-engine.ts
Token Decision Engine	Decisiones per-token: OPERATE/SKIP/WATCH/EXIT/ADJUST	token-decision-engine.ts
Kill Switch Service	7 kill switches, concentracion, auto-pause, manual controls	kill-switch-service.ts
Risk Controls Verifier	Pre-trade: limites de posicion, DD, daily loss, kill switch	risk-controls-verifier.ts
Operability Score	Score 0-100, niveles PREMIUM->UNOPERABLE, fee-aware	operability-score.ts

Modulo	Funcion	Archivo
Capital Allocation Engine	16 metodos (5 activos v1), position sizing	capital-allocation.ts
Capital Strategy Manager	Modo ULTRA_CONSERVATIVE->CONCENTRATED, learning state	capital-strategy-manager.ts
Monte Carlo Simulator	Simulacion de riesgo, VaR, CVaR, max DD distribution	monte-carlo-simulator.ts
Alert Engine	6 categorias, 3 severidades, distribucion WS	alert-engine.ts
Strategy Correlation	Matrices de correlacion, deteccion sobre-concentracion	strategy-correlation-service.ts
Data Quality Gate	Compleitud, frescura, consistencia, precision	data-quality-gate.ts
Regime Heuristic	5 regimenes: TRENDING_UP/DOWN, SIDEWAYS, HIGH/LOW_VOL	regime-heuristic.ts
Data Retention	Políticas de retencion por tipo, archival, cleanup	data-retention.ts
Buy/Sell Pressure	Presion desde DEX swaps, SM vs retail, senales	buy-sell-pressure.ts

3.5 Fase 5: Asignacion de Capital

El Capital Allocation Engine implementa 16 metodos de asignacion, de los cuales 5 estan activos en v1: Kelly Modified (default single-strategy), Risk Parity (default multi-strategy), Volatility Targeting (regimen alta vol), Max Drawdown Control (DD>10%), y Equal Weight (fallback). El SDE selecciona automaticamente el metodo basandose en el estado del portfolio, regimen de mercado, y drawdown actual. Los 11 metodos restantes estan deprecados o planificados para v2. El Capital Strategy Manager gestiona el modo de operacion (ULTRA_CONSERVATIVE a CONCENTRATED) y mantiene estado de aprendizaje persistente entre reinicios.

3.6 Fase 6: Ejecucion

El Paper Trading Engine opera como simulador de trading con posiciones virtuales, tracking de PnL, y estadisticas completas (win rate, Sharpe, max drawdown). Soporta start/stop/pause y gestion de posiciones. El Trade Execution Engine proporciona controles pre-trade (risk checks), creacion de ordenes, routing DEX (Jupiter/1inch/ParaSwap), y limites diarios. El Autonomous Execution Engine procesa senales, crea ordenes, y gestiona posiciones de forma autonoma con integracion completa con kill switches y capital strategy.

Modulo	Funcion	Archivo
Paper Trading Engine	Posiciones virtuales, PnL, stats, start/stop/pause	paper-trading-engine.ts
Trade Execution Engine	Pre-trade risk checks, ordenes, DEX routing, limites diarios	trade-execution-engine.ts
Autonomous Execution	Loop autonomo: senal->orden->posicion->portfolio	autonomous-execution-engine.ts
Trade Executor Arch	Interfaces y contratos DEX, tipos de orden	trade-executor-arch.ts

4. Catalogo de Modulos

El sistema contiene 89 archivos de servicio organizados en 6 directorios bajo src/lib/services/, mas 10 archivos de soporte en src/lib/ raiz. Se identifican aproximadamente 40 singletons activos que proporcionan funcionalidad a traves de toda la aplicacion. A continuacion se presenta el resumen por directorio.

Directorio	Archivos	Singletons Clave
services/brain/	13	brainScheduler, brainCycleEngine, brainCapacityEngine, phaseStrategyEngine, candlestickPatternEngine, tokenLifecycleEngine, behavioralModelEngine, patternCompressionPipeline
services/execution/	9	tradeExecutionEngine, smartMoneyTracker, autonomousExecutionEngine, paperTradingEngine, botDetection
services/strategy/	16	strategyStateManager, deepAnalysisEngine, strategyEvolutionEngine, decisionEngine (TDE), strategyDecisionEngine (SDE), autoEvolutionLoop, regimeHeuristic, tradingSystemEngine
services/risk/	14	crossCorrelationEngine, strategyCorrelationService, monteCarloSimulator, killSwitchService, capitalAllocationEngine, capitalStrategyManager, alertEngine, operabilityScore
services/data-sources/	14	coinGeckoClient, dexScreenerClient, dexPaprikaClient, binanceClient, etherscanClient, ohlcvPipeline, realDataLoader
services/backtesting/	7	backtestingEngine, feedbackLoopEngine, backtestLoopEngine, walkForwardEngine, statisticalValidation
services/shared/	6	sharedClients, rateLimiter, requestSemaphore, universalExtractor, userDataFilter
lib/ (root)	10	db, wsBridge, unifiedCache, requestQueue, validations, format, startup
TOTAL	89	~40 singletons activos



Figura 3: Mapa de dependencias entre modulos — Líneas gruesas = dependencias clave, finas = estandar, discontinuas = cross-group

5. Mapa de Rutas API

El sistema expone 112 endpoints API organizados en 11 categorías funcionales. Todas las rutas siguen el patron Next.js App Router bajo /src/app/api/. A continuacion se presenta el catalogo completo por categoria.

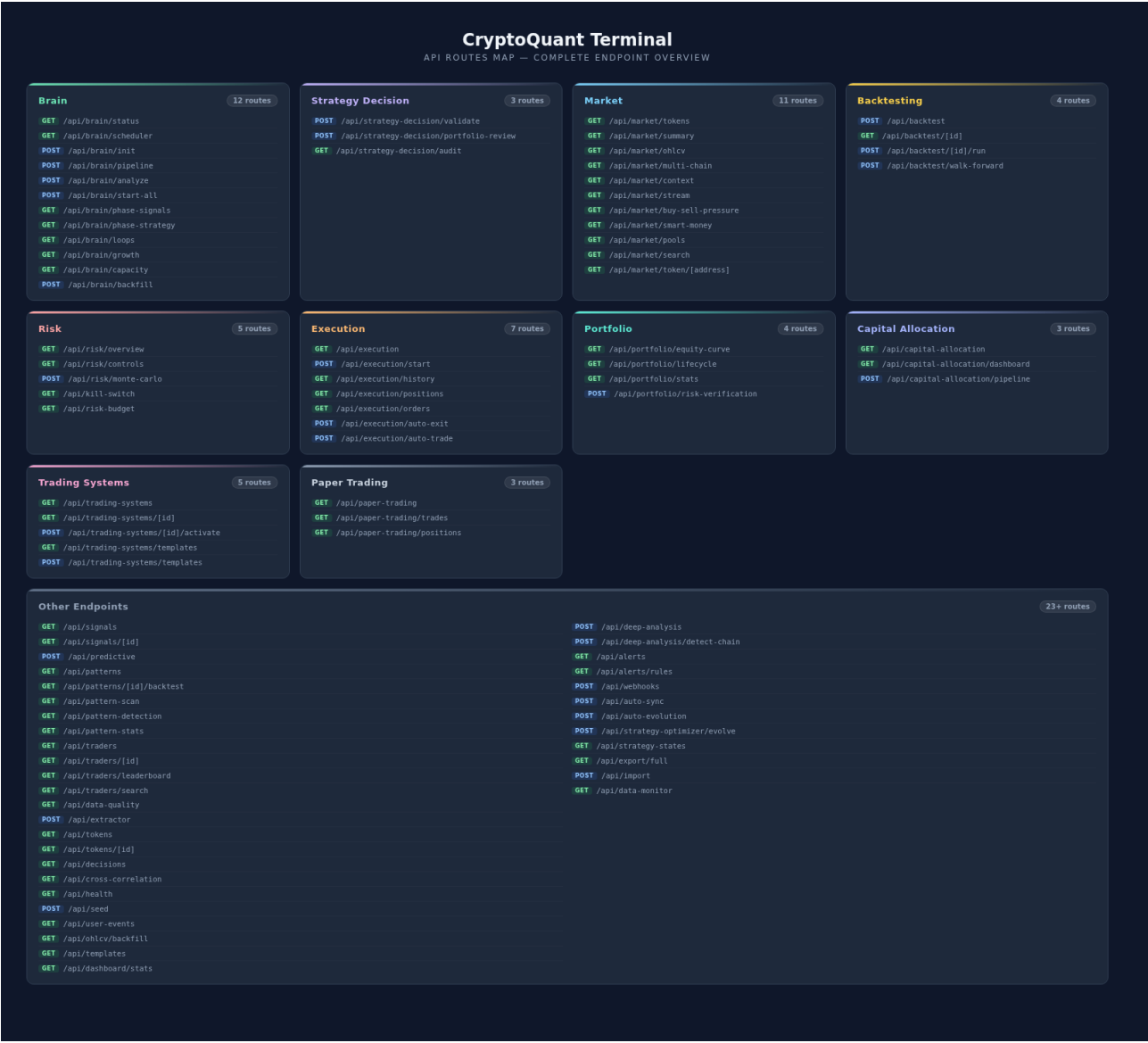


Figura 4: Mapa de rutas API organizadas por categoria funcional

Categoria	Count	Rutas
Brain	12	/api/brain/{status, scheduler, init, pipeline, analyze, start-all, phase-signals, phase-strategy, loops, growth, capacity, backfill}
Strategy Decision	3	/api/strategy-decision/{validate, portfolio-review, audit}
Market	11	/api/market/{tokens, summary, ohlcv, multi-chain, context, stream, buy-sell-pressure, smart-money, pools, search, token/[addr]}
Backtesting	4	/api/backtest{, /[id], /[id]/run, /walk-forward}
Risk	5	/api/risk/{overview, controls, monte-carlo} + /api/{kill-switch, risk-budget}
Execution	7	/api/execution{, /start, /history, /positions, /orders, /auto-exit, /auto-trade}
Portfolio	4	/api/portfolio/{equity-curve, lifecycle, stats, risk-verification}

Categoria	Count	Rutas
Capital	3	/api/capital-allocation{, /dashboard, /pipeline}
Trading Systems	5	/api/trading-systems{, /[id], /[id]/activate, /templates}
Paper Trading	3	/api/paper-trading{, /trades, /positions}
Other	23+	/api/{signals, predictive, patterns, traders, deep-analysis, alerts, webhooks, auto-sync, auto-evolution, strategy-optimizer, strategy-evolution, strategy-states, export/*, import, data-monitor, data-quality, extractor, tokens, decisions, regime, templates, ohlcv/backfill, cross-correlation, health, seed, user-events}

6. Esquema de Base de Datos

El sistema utiliza SQLite como base de datos a través de Prisma 7. El esquema contiene 38 modelos con 22 tablas mapeadas explícitamente via `@@map`. No se utilizan enums formales de Prisma — todos los valores tipo-enum son Strings con valores documentados en comentarios. La base de datos es altamente JSON-heavy, con 40+ campos almacenando JSON strings (SQLite no tiene tipo nativo JSON). A continuación se presentan los modelos organizados por dominio funcional.

Modelos por Dominio

Dominio	Count	Modelos Clave
Usuario y Auth	2	User (8 campos), rol USER/ADMIN
Tokens (Multi-chain)	2	Token (20+ campos), PriceCandle (OHLCV + source + unique composite)
Trader Intelligence	5	Trader (55+ campos, 8 secciones), TraderTransaction, WalletTokenHolding, TraderBehaviorPattern (22 tipos), CrossChainWallet (7 link types)
Señales y Analisis	5	Signal, UserEvent, PatternRule, TokenDNA (1:1 con Token), PredictiveSignal (11 tipos)
Trading Systems Lab	3	TradingSystem (5-layer config, self-ref parentSystemId), BacktestRun (anti-overfitting fields), BacktestOperation
Brain Cycle (24/7)	4	BrainCycleRun, OperabilitySnapshot (6 sub-scores), CompoundGrowthTracker, TradingCycle
Behavioral/Learning	3	TraderBehaviorModel (unique archetype+phase+action), FeedbackMetrics, SystemEvolution
Paper Trading	3	PaperTradingSession, PaperTradingPosition, PaperTradingTrade
Alerts & Notif	3	AlertRule (6 categorías, 3 severidades), Alert, WebhookConfig
Risk Controls	3	RiskBudget (7 límites, 3 perfiles), RiskControlsConfig, DecisionAudit
Estrategia/Evolucion	3	StrategyStateHistory (7 estados), EvolutionCycle (6 fases), AIBestStrategy (Hall of Fame)
Extraccion/Jobs	4	ExtractionJob, DataRetentionPolicy, ApiRateLimit, DecisionLog
Otros	3	SchedulerState, ProtocolData, StrategyTemplate (marketplace)

Relaciones Principales

User -> TradingSystem[], BacktestRun[], PaperTradingSession[], PaperTradingPosition[], AlertRule[], Alert[], WebhookConfig[], StrategyTemplate[]

Token -> TokenDNA? (1:1), Signal[], PriceCandle[], TokenLifecycleState[]

Trader -> TraderTransaction[], WalletTokenHolding[], TraderBehaviorPattern[], CrossChainWallet[] (x2: Primary+Linked), TraderLabelAssignment[]

TradingSystem -> BacktestRun[], BacktestOperation[], TradingSystem[] (self-ref: parentSystemId -> derivedSystems)

BacktestRun -> BacktestOperation[]

DecisionAudit -> strategyId es String plano (NO tiene @relation formal a TradingSystem — discrepancia con el doc de arquitectura)

Observaciones del Schema

- **No hay Enums Prisma:** Todos los valores tipo-enum son Strings. Reduce type safety pero facilita migración.
- **userId nullable:** Muchos modelos tienen userId String? "para backward compat" — indica transición de single-user a multi-user.

- **40+ campos JSON:** SQLite no tiene tipo nativo JSON. Parsing/validation ocurre en la capa de aplicacion.
- **DecisionAudit.strategyId sin FK formal:** El doc de arquitectura especifica relacion con TradingSystem, pero el schema Prisma usa String plano.
- **Dos modelos de operability:** OperabilityScore y OperabilitySnapshot coexisten con solapamiento.
- **No hay migraciones formales:** Se usa prisma db push (prototyping), no prisma migrate dev.

7. Integraciones Externas

El sistema se integra con 13+ fuentes de datos externas, cada una con su propio cliente especializado, rate limiting, y cache. Las integraciones se organizan en 3 categorias: fuentes de datos de mercado (precios, volúmenes, OHLCV), fuentes on-chain (transacciones, wallets, smart money), y fuentes de protocolos DeFi (TVL, yields). Además, el sistema utiliza z-ai-web-dev-sdk para LLM integration en el Deep Analysis Engine, y Socket.IO para comunicacion real-time entre backend y frontend.

Fuente	Tipo	Auth	Datos	Cache
CoinGecko	Mercado	Gratuita	Tokens, OHLCV, global data, trending	Si (60s-5min)
DexScreener	Mercado	Gratuita	DEX pairs, liquidez, trending	Si (30s-2min)
DexPaprika	Mercado	Gratuita	20+ chains, pools, swaps, OHLCV, SM swaps	Si (60s-5min)
Binance	Mercado	Gratuita	OHLCV candles, tickers, symbol info	Si (15s-1min)
Etherscan	On-chain	API Key	Token transfers, trader discovery	Si (rate-limited)
Footprint	Mercado	Gratuita	Token prices, protocol TVL, chain overview	Si (5min)
SQD/Subsquid	On-chain	Gratuita	Indexer blockchain, eventos historicos	On-demand
Helius	On-chain	Gratuita	Transacciones Solana	Si (2min)
Moralis	On-chain	Gratuita	Multi-chain wallets, balances	Si (5min)
DefiLlama	DeFi	Gratuita	Protocol TVL, yields, chain overview	Si (10min)
CryptoDataDownload	Historico	Gratuita	CSV historical data	On-demand
Dune	On-chain	Gratuita	SQL queries on blockchain data	On-demand
z-ai-web-dev-sdk	LLM	SDK interno	Chat completions para Deep Analysis	N/A
Socket.IO	Real-time	Interno	WS bridge backend->frontend (puerto 3010)	N/A

Cascada OHLCV

El OHLCV Pipeline implementa una cascada de 3 niveles para maximizar la cobertura de datos: **Nivel 1 (Binance)**: Mayor cobertura, datos mas precisos, intervalos de 1m a 1M. Se usa como fuente primaria para tokens listados en Binance. **Nivel 2 (CoinGecko)**: Cobertura amplia de tokens CEX, OHLCV con timeframes limitados. Fallback cuando Binance no tiene el token. **Nivel 3 (DexPaprika)**: Tokens DEX no listados en CEX, cobertura de 20+ chains. Ultimo recurso para tokens esotericos o nuevos. El pipeline ademas agrega timeframes superiores automaticamente (1m->5m->15m->1h->4h->1d) y maneja backfill historico con deteccion de gaps.

8. Pipeline del Strategy Decision Engine (SDE)

El SDE es el componente mas critico del sistema. Transforma datos de multiples fuentes (backtest, Monte Carlo, walk-forward, operability, paper trading, regime) en una decision accionable con 3 dimensiones. Implementa el principio de "vetos antes que scores": un veto duro SIEMPRE tiene prioridad sobre un score alto, sin excepciones. El pipeline es deterministicamente reproducible — mismos inputs producen siempre el mismo output.

Entrada: SDEInput

Campo	Tipo	Datos Clave
backtest	BacktestSnapshot	totalTrades, winRate, avgWinPct, avgLossPct, maxDrawdownPct, sharpeRatio, payoffRatio, overfittingScore, parameterStability
monteCarlo	MonteCarloSnapshot	riskOfRuin, probabilityOfProfit, p95MaxDrawdown, meanFinalEquity, simulationsCount
walkForward	WalkForwardSnapshot	aggregateWFE, isRobust, parameterStability, overallDegradation, performanceConsistency, windowCount
operability	OperabilitySnapshot	overallScore, level, isOperable, recommendedPositionUsd, minimumGainPct, feeEstimateTotalCostPct
paperTrading	PaperTradingSnapshot?	totalTrades, winRate, unrealizedPnlPct, currentDrawdownPct, daysActive, sharpeRatio
portfolioState	Object	totalCapitalUsd, currentDrawdownPct, activeStrategies, marketVolatility, marketRegime
regimeAssessment	Object?	regime, confidence, volatilityPercentile, trendDirection, trendStrength

Salida: StrategyDecision

Campo	Tipo	Descripcion
state	StrategyState	ACTIVE CONDITIONAL PAUSED REJECTED
capitalAction	CapitalAction	INCREASE MAINTAIN REDUCE EXIT
signalQuality	SignalQuality	STRONG ADEQUATE WEAK
scores	CompositeScores	robustness (0-100), overfitting (0-100, lower=better), stability (0-100)
vetoResults	VetoResult[]	5 vetos con passed/value/threshold/reason
capitalRecommendation	CapitalRecommendation	targetPct, sizeUsd, method, reason
recommendations	string[]	Sugerencias adicionales contextuales
auditId	string	Referencia al registro de auditoria completo

Pesos Fijos v1

Score	Formula
Robustness	WFE x 0.35 + probOfProfit x 0.30 + paramStability x 0.35
Overfitting	degradation x 0.45 + WFE_variance x 0.30 + tradeCountPenalty x 0.25
Stability	paramStability x 0.40 + OOS_winRate_std x 0.30 + regimeConsistency x 0.30

Umbrales de Veto por Perfil de Riesgo

Veto	CONSERVATIVE	MODERATE	AGGRESSIVE
MIN_TRADES	50	50	50
MAX_RISK_OF_RUIN	1%	3%	5%
MAX_DRAWDOWN	30%	40%	50%
MIN_WFE	40%	30%	25%
MIN_WIN_RATE (w/ low payoff)	40%	35%	30%

9. Kill Switches y Gestion de Riesgo

El Kill Switch Service implementa 7 niveles de proteccion de capital, cada uno con umbrales configurables cargados desde la tabla RiskBudget de la base de datos. Cuando un kill switch se activa, se genera automaticamente una alerta via el Alert Engine y se propaga via WS Bridge al frontend. Los kill switches operan con histeresis del 80% para evitar activacion/desactivacion rapida — un kill switch activado por DD>20% solo se desactiva cuando DD baja del 16%.

Kill Switch	Campo RiskBudget	Default	Accion	Descripcion
Portfolio DD	maxPortfolioDrawdownPct	20%	PAUSE_ALL	Auto-pause todo el trading
Strategy DD	maxStrategyDrawdownPct	30%	PAUSE_STRATEGY	Auto-pause estrategia especifica
Position Loss	maxPositionLossPct	50%	CLOSE_POSITION	Auto-close posicion con perdida excesiva
Token Concentration	maxConcentrationPct	15%	REJECT_POSITION	Rechazar trade que exceda concentracion
Chain Concentration	maxChainPct	50%	REJECT_POSITION	Rechazar trade por concentracion de cadena
Sector Concentration	maxSectorPct	30%	REJECT_POSITION	Rechazar trade por concentracion sectorial
Emergency Global Pause	N/A (manual)	N/A	PAUSE_ALL	Pause manual de emergencia

Flujo de canOpenPosition()

Antes de abrir cualquier posicion, el sistema ejecuta 7 comprobaciones secuenciales: (1) Global manual pause activo? -> Rechazar. (2) Per-strategy manual pause? -> Rechazar. (3) Portfolio DD kill switch activo? -> Rechazar. (4) Strategy DD kill switch activo? -> Rechazar. (5) Token concentration excederia limite? -> Rechazar. (6) Chain concentration excederia limite? -> Rechazar. (7) Sector concentration excederia limite? -> Rechazar. Solo si todas pasan, se permite la apertura. Este flujo garantiza que ninguna posicion se abra sin verificar todos los limites de riesgo configurados.

10. Capital Allocation Pipeline

El pipeline de asignacion de capital conecta el SDE con el Paper Trading Engine a traves del Capital Allocation Engine. El flujo es: SDE produce StrategyDecision con capitalRecommendation -> CapitalAllocationEngine calcula tamanos de posicion usando el metodo seleccionado -> Strategy Correlation Service verifica que no se excedan limites de correlacion -> Kill Switch Service verifica limites de concentracion -> Paper Trading Engine ejecuta con el tamaño aprobado.

5 Metodos Activos en v1

Metodo	Cuando se usa	Formula	Rationale
KELLY_MODIFIED	Default single-strategy	Half-Kelly: $f^* = (p \cdot b - q) / b$	Maximiza crecimiento a largo plazo con riesgo controlado

Metodo	Cuando se usa	Formula	Rationale
RISK_PARITY	Default multi-strategy (2+)	w_i proporcional a $1/\sigma_i$	Cada estrategia contribuye igual riesgo
VOLATILITY_TARGETING	Alta vol (regimen/percentil 75)	$size = baseSize \times (targetVol / realizedVol)$	Reduce exposicion cuando vol sube
MAX_DRAWDOWN_CONTROL	DD > 10% o REDUCE action	$size = baseSize \times (1 - currentDD / maxDD)$	Protege capital durante rachas negativas
EQUAL_WEIGHT	Fallback/benchmark	$size = capital / numAssets$	Simple, predecible, punto de referencia

Seleccion Automatica de Metodo

El SDE selecciona automaticamente el metodo de asignacion basandose en el contexto del portfolio: (1) Si DD > 10% o capitalAction = REDUCE -> MAX_DRAWDOWN_CONTROL. (2) Si avg pairwise correlation > 0.5 -> RISK_PARITY. (3) Si regimen = HIGH_VOLATILITY -> VOLATILITY_TARGETING. (4) Si regimen = TRENDING_DOWN y DD > 5% -> MAX_DRAWDOWN_CONTROL. (5) Si marketVolatility > percentil 75 -> VOLATILITY_TARGETING. (6) Si activeStrategies >= 2 -> RISK_PARITY. (7) Default -> KELLY_MODIFIED (single strategy, buenas condiciones). Despues del calculo, se aplica un hard constraint de concentracion max 15% por estrategia.

11. Estado de las Conexiones Criticas

Esta seccion documenta el estado real de las conexiones entre modulos, identificando cuales estan operativas, cuales son parciales, y cuales faltan completamente. Este analisis es fundamental para la Fase 4 (Auditoria de Integraciones) y la Fase 6 (Correcciones).

Conexion	Estado	Detalle
OHLCV Pipeline -> Brain Orchestrator	OPERATIVA	El Brain importa y usa ohlcvPipeline directamente
Brain Scheduler -> Brain Cycle Engine	OPERATIVA	Lazy import, loop SCAN->FILTER->MATCH->STORE
Brain Cycle -> Feedback Loop Engine	PARCIAL	Importado pero NUNCA llamado (feedbackLoopEngine no se invoca en brainCycleEngine)
Backtest Engine -> SDE	OPERATIVA	SDE lee BacktestSnapshot de backtest results via buildInputFromStrategyId
Monte Carlo -> SDE	PARCIAL	SDE acepta MonteCarloSnapshot pero buildInputFromStrategyId genera PLACEHOLDER (no corre MC real)
Walk-Forward -> SDE	PARCIAL	Igual que MC: datos placeholder si no se corre WF explicitamente
SDE -> Paper Trading	NO CONECTADA	PTE tiene su propia logica de decision inline, no usa SDE
SDE -> Capital Allocation	OPERATIVA	SDE llama capitalAllocationEngine.calculate() en Step 5
Capital Allocation -> Paper Trading	NO CONECTADA	PTE usa calculatePositionSize() propio (equal-split), no usa CapitalAllocationEngine
Kill Switch -> Paper Trading	OPERATIVA	PTE verifica kill switches antes de abrir posiciones
Kill Switch -> Trade Execution	OPERATIVA	Trade Execution verifica kill switches en pre-trade checks
Alert Engine -> WS Bridge	OPERATIVA	Alert Engine llama wsBridge.pushAlert() para notificaciones real-time
Auto-Evolution Loop -> SDE	NO CONECTADA	Auto-evolution activa estrategias por minSharpeRatio, no pasa por SDE
Feedback Loop -> Auto-Evolution	OPERATIVA	Auto-evolution importa feedbackLoopEngine para evaluacion
Strategy Correlation -> SDE	OPERATIVA	SDE llama strategyCorrelationService para verificacion de correlacion
Regime Heuristic -> SDE	PARCIAL	SDE acepta regimeAssessment opcional, pero no lo genera automaticamente
Paper Trading -> SDE (feedback)	NO CONECTADA	No hay mecanismo automatico para feedear resultados de PTE al SDE
TDE <-> SDE	NO INTEGRADOS	Token Decision Engine y Strategy Decision Engine coexisten sin integracion
Risk Controls Verifier -> Paper Trading	HARDCODED	RiskControlsVerifier usa valores hardcoded, no hace analisis real

Resumen de Estado de Conexiones

Estado	Count	Descripcion
OPERATIVAS	9	Conexiones funcionando correctamente
PARCIALES	4	Conectadas pero con datos placeholder o sin uso completo
NO CONECTADAS	5	Modulos que deberian estar conectados pero no lo estan
HARDCODED	1	Existe pero usa logica hardcoded en vez de analisis real

12. Hallazgos y Observaciones

A continuacion se presentan los hallazgos mas significativos del mapeo completo del sistema. Estos hallazgos seran la base para las fases posteriores de la auditoria. **No se proponen correcciones en esta fase** — las correcciones se realizaran exclusivamente en la Fase 6.

12.1 Gaps Arquitectonicos Criticos (confirmados)

Se confirman 6 gaps arquitectonicos previamente identificados que representan las desconexiones mas importantes entre el diseno del sistema (ARCHITECTURE_FINAL.md) y la implementacion real:

Prioridad	Gap	Descripcion
P0	Auto-feedback loop (S1.14)	feedbackLoopEngine importado pero nunca llamado en brainCycleEngine. El loop SCAN->...->FEEDBACK no ejecuta el paso FEEDBACK.
P0	SDE->PTE full integration (S1.13)	Paper Trading Engine tiene logica de decision inline propia, no usa SDE para decidir que estrategias operar.
P1	maxDailyVaR en RiskBudget	Ausente del schema Prisma y del servicio. El doc de arquitectura lo menciona pero no esta implementado.
P1	Alert escalation chain (S2.8)	No hay cadena INFO->WARNING->CRITICAL->AUTO_PAUSE. Las alertas se generan pero no escalan automaticamente.
P1	TDE<->SDE integration	Token Decision Engine y Strategy Decision Engine coexisten sin integracion. TDE decide por token, SDE por estrategia, sin comunicacion.
P2	RiskControlsVerifier real analysis	RiskControlsVerifier usa valores hardcoded en vez de hacer analisis real de riesgo.

12.2 Observaciones de Data Flow

- **MC/WF datos placeholder:** Cuando buildInputFromStrategyId() no encuentra resultados reales de Monte Carlo o Walk-Forward, genera datos placeholder (fabricados). El SDE marca esto como dataQuality=PLACEHOLDER y aplica logica conservadora (max CONDITIONAL + MAINTAIN), pero el usuario podria no ser consciente de que las decisiones se basan en datos fabricados.
- **Capital Allocation huérfano:** El Capital Allocation Engine tiene 16 metodos implementados pero solo el SDE lo llama (para calcular capitalRecommendation). El Paper Trading Engine usa su propio calculatePositionSize() que divide capital equitativamente entre posiciones, ignorando completamente el motor de allocation.
- **Regime no conectado automaticamente:** El RegimeHeuristic existe y funciona, pero el SDE solo lo usa si alguien le pasa regimeAssessment en el input. No hay mecanismo automatico que ejecute el regimen heuristic y alimente su resultado al SDE.
- **Feedback loop roto:** El SDE tiene provideFeedback() para actualizar audit records con resultados, pero nada lo llama automaticamente. Cuando paper trading cierra una posicion con PnL, nadie feedea ese resultado al SDE para reevaluacion.

12.3 Observaciones de Schema

- **DecisionAudit sin FK formal:** strategyId es String plano sin @relation a TradingSystem. El doc de arquitectura especifica una relacion formal, pero el schema no la implementa.

- **Dos modelos de operability:** OperabilityScore y OperabilitySnapshot tienen campos solapados (ambos calculan score, nivel, isOperable). No esta claro cual es la fuente de verdad.
- **userId nullable extendido:** Muchos modelos tienen userId String? "para backward compat". Esto sugiere que el sistema fue disenado inicialmente como single-user y se esta migrando a multi-user, pero la migracion no esta completa.

12.4 Observaciones de UI

- **19 pestanas activas:** La UI tiene 19 tabs organizados en 5 grupos (Market, Intelligence, Strategy, Risk & Portfolio, Tools). Cada tab es un componente complejo con multiples sub-componentes.
- **Zustand store con 26 slices:** useCryptoStore gestiona 26 slices de estado, desde tokens y senales hasta filtros y configuracion. El store se actualiza via WS y REST polling.
- **Socket.IO real-time:** WebSocketProvider en el frontend se conecta al servidor Socket.IO en puerto 3010 con fallback a REST polling. SimulationProvider genera datos simulados cuando WS no esta disponible.

12.5 Observaciones de Configuracion

- **Next.js 16 con ignoreBuildErrors:** typescript.ignoreBuildErrors=true en next.config.ts. El build compila pero podria ocultar errores de tipos.
- **noImplicitAny: false:** TypeScript configurado sin verificacion implicita de any. Reduce seguridad de tipos.
- **reactStrictMode: false:** Modo estricto de React desactivado. Podria ocultar bugs de efectos.
- **Auth deshabilitado:** AUTH_ENABLED=false por defecto. El sistema usa un demo user automatico via getCurrentUserId().

Fin de la Fase 1: Auditoria de Flujo Completo. No se han realizado modificaciones al codigo. Proxima fase: Fase 2 — Comparacion con el documento de reestructuracion.